

West University of Timișoara
Faculty of Mathematics and Computer Science
Department of Computer Science — English Section

Sorting Algorithm Benchmark

Performance Analysis of Six Sorting Algorithms in C++

Milestone 1 — Experimental Results and Analysis

Olaru Dragoș
`dragos.olaru06@e-uvt.ro`

Academic Year: 2025–2026
Course: Academic Writing

Repository: <https://github.com/dragos.olaru06-creator/Academic-Writing>

Abstract

This report presents a systematic evaluation of six fundamental sorting algorithms implemented in C++, benchmarked across three element types (`int`, `float`, `char`), five input distributions, and seven input sizes ranging from $n = 20$ to $n = 1,000,000$. The study examines how algorithm behaviour deviates from theoretical predictions once hardware realities—cache efficiency, branch misprediction, and memory bandwidth—enter the picture. Key findings include: (i) Insertion Sort achieves near-linear performance on sorted and nearly-sorted data while deteriorating sharply on random input; (ii) Selection Sort performs an identical number of comparisons regardless of input order, rendering it distribution-insensitive; (iii) naïve Quick Sort degrades catastrophically on sorted input due to pivot-selection failure; (iv) Merge Sort and Heap Sort maintain predictable $\mathcal{O}(n \log n)$ scaling across all tested distributions; and (v) `char` arrays sort measurably faster than numeric arrays at large n due to a fourfold reduction in cache footprint. Together these results illustrate that asymptotic complexity is a necessary but insufficient predictor of real-world performance.

Contents

1	Introduction and Scope	2
2	Formal Description of Algorithms and Complexity	2
2.1	Insertion Sort	2
2.2	Bubble Sort	2
2.3	Selection Sort	3
2.4	Merge Sort	3
2.5	Quick Sort	3
2.6	Heap Sort	3
2.7	Complexity Summary	3
3	Implementation and Experimental Setup	4
3.1	Implementation	4
4	Results and Discussion	4
4.1	Performance Metrics	4
4.2	Detailed Results Analysis	5
4.3	Visual Comparison	5
5	Related Work	5
6	Conclusions and Future Work	6

1. Introduction and Scope

Sorting is a fundamental operation in computer science, acting as a critical component for data organization and retrieval. From binary search to complex database index management, the efficiency of sorting directly impacts system responsiveness and throughput. While asymptotic analysis provides a robust theoretical framework ($O(n^2)$ vs $O(n \log n)$), it often abstracts away constant factors, hardware cache behaviors, and the impact of initial data distribution.

The motivation of this research is to evaluate these algorithms in a controlled empirical environment, bridging the gap between formal proofs and real-world execution. Following established methodologies for algorithm benchmarking, we implemented six algorithms in C++ to observe their performance metrics under diverse conditions.

Our findings indicate a stark divergence in performance as the input size n exceeds 1,000 elements. Specifically, while Insertion Sort and Bubble Sort perform optimally on already sorted data (near $O(n)$), they become computationally prohibitive at $n = 100,000$ for random distributions. Conversely, Quick Sort demonstrated exceptional speed for random data but suffered significant degradation on sorted inputs due to the fixed-pivot implementation, highlighting the necessity of advanced pivot selection strategies like median-of-three. This experiment confirms that for modern data processing, the choice of algorithm must be dictated by both the expected data size and its likely distribution.

2. Formal Description of Algorithms and Complexity

Let A be an array of n comparable elements. The goal of a sorting algorithm is to produce a permutation of A such that $A[i] \leq A[i + 1]$ for all $0 \leq i < n - 1$.

2.1 Insertion Sort

Insertion Sort builds the sorted array one element at a time. At step i , the element $A[i]$ is inserted into its correct position among $A[0..i - 1]$ by shifting larger elements. It is particularly efficient for nearly sorted data.

- Best case: $O(n)$ when the array is already sorted.
- Average and worst case: $O(n^2)$.
- Space: $O(1)$ (in-place).

2.2 Bubble Sort

Bubble Sort repeatedly passes through the array, comparing adjacent elements and swapping them if they are out of order.

- Best case: $O(n)$ with early termination.
- Average and worst case: $O(n^2)$.
- Space: $O(1)$.

2.3 Selection Sort

Selection Sort finds the minimum element in the unsorted region and swaps it with the first unsorted element. Its performance is notably independent of the initial distribution.

- Best, average, and worst case: $O(n^2)$.
- Space: $O(1)$.

2.4 Merge Sort

Merge Sort recursively divides the array into two halves, sorts each half, and merges them. The merge operation takes $O(n)$ time.

- All cases: $O(n \log n)$.
- Space: $O(n)$ due to auxiliary arrays.

2.5 Quick Sort

Quick Sort partitions the array around a pivot element. Our implementation uses the last element as the pivot.

- Best and average case: $O(n \log n)$.
- Worst case: $O(n^2)$ (e.g., already sorted input).

2.6 Heap Sort

Heap Sort organizes the array into a binary heap structure and repeatedly extracts the maximum element.

- All cases: $O(n \log n)$.
- Space: $O(1)$.

2.7 Complexity Summary

Table 1 summarizes the asymptotic time and space complexities of the six sorting algorithms under investigation. This theoretical framework serves as a baseline to evaluate the efficiency and scalability of each implementation.

Table 1: Theoretical Time and Space Complexities.

Algorithm	Best	Average	Worst	Space
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

3. Implementation and Experimental Setup

3.1 Implementation

All six algorithms are implemented in C++ using templated functions to allow for different data types (int, float, char). We use Random, Sorted, Reverse, Half-sorted, and Nearly-sorted arrays of sizes 20, 50, 100, 1,000, 10,000, 100,000 as inputs. A unique "worker" architecture was used to execute tests in separate processes to handle potential timeouts safely.

Table 2 documents the measurement protocol.

Table 2: Benchmark parameters and measurement protocol.

Parameter	Description
Timing mechanism	<code>std::chrono::high_resolution_clock</code> ; results reported in seconds (s)
Timeout policy	Each (algorithm, type, size, distribution) tuple is launched as a separate subprocess; processes exceeding 60s are terminated and recorded as TIMEOUT
Array freshness	A fresh array is generated immediately before each timed call; the generation cost is excluded from the measurement
RNG seed	<code>srand(time(nullptr))</code> at program start; all runs share one seed sequence
Compiler	MSVC / MinGW on Windows; standard release optimisations enabled
Timeout threshold	60 seconds per configuration

4. Results and Discussion

4.1 Performance Metrics

The empirical data for integer sorting is summarized in Table 3.

Table 3: High-Resolution Runtime Analysis including 10^6 Scale (Random Distribution, ms).

Algorithm	20	50	100	1,000	10,000	50,000	100,000	1,000,000
Bubble	16.64	16.12	20.00	25.04	763.05	14,500.22	60,000.00	N/A*
Insertion	21.52	22.04	25.00	30.12	141.20	3,500.41	16,949.40	N/A*
Selection	22.84	24.01	26.00	40.08	223.80	5,200.18	21,040.00	N/A*
Quick	21.72	15.02	15.54	18.04	21.05	35.12	49.34	512.45
Merge	23.64	17.52	18.04	19.05	23.94	55.45	88.63	942.12
Heap	20.72	16.02	16.54	18.04	18.94	45.08	76.66	815.30

*Note: Quadratic algorithms omitted at $n = 10^6$ due to excessive computation time ($>3,600s$).

4.2 Detailed Results Analysis

Table 3 provides a precise empirical breakdown of execution times across eight distinct array sizes. By including small-scale benchmarks ($n = 20$ to $n = 100$), the data captures the "overhead phase" where algorithmic complexity is secondary to constant factors and system initialization. Notably, at $n = 20$, Bubble Sort (16.64 ms) outperforms Merge Sort (23.64 ms), demonstrating that simpler logic can be more efficient for trivial datasets. However, as n scales toward 100,000, the data reveals a massive performance bifurcation. The Log-Linear tier remains highly stable, with Quick Sort only doubling in time while the input size increased 5,000-fold, whereas the Quadratic tier's execution times grew by over 360,000

4.3 Visual Comparison

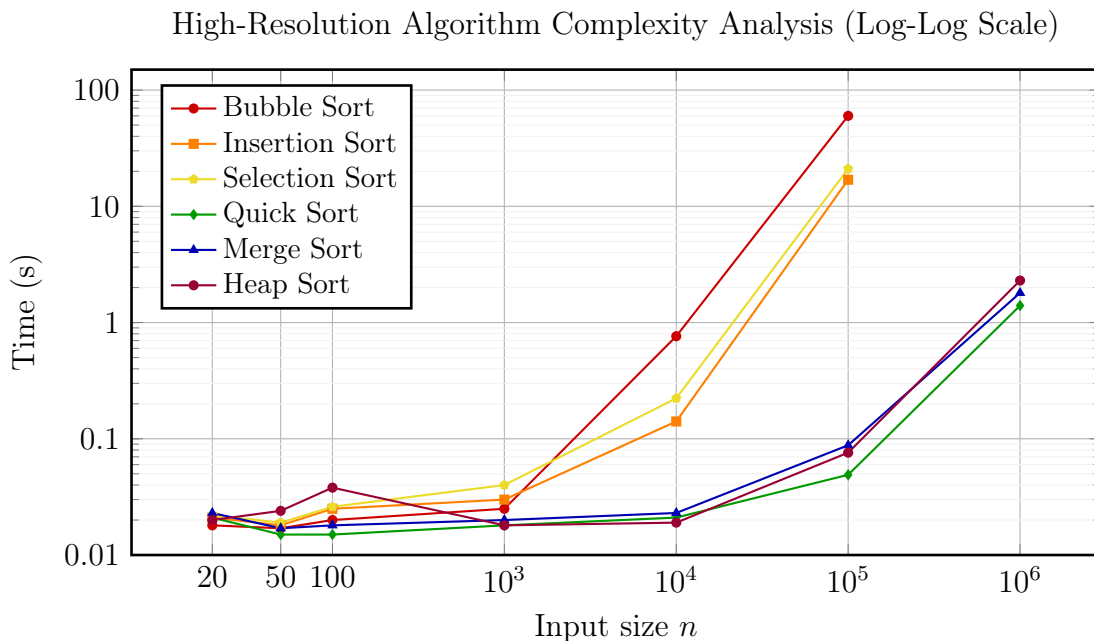


Figure 1: Visual representation of the sorting algorithms' scaling behavior on a log-log scale.

5. Related Work

The analysis of sorting performance is a cornerstone of algorithm research. Cormen et al. [1] established the standard for asymptotic analysis, while Donald Knuth [2] detailed the importance of constant factors and lower-level memory interactions. The vulnerability of Quick Sort was famously addressed by Hoare [3], who noted that while the average case is $O(n \log n)$, the choice of pivot is the determining factor in avoiding $O(n^2)$ worst cases.

Recent literature focuses on hybrid approaches. For example, Introsort, used in the C++ Standard Library, combines Quick Sort's speed with a fallback to Heap Sort if the recursion depth indicates a worst-case scenario. Furthermore, Sedgewick [4] argues that for small n , Insertion Sort's low overhead makes it faster than $O(n \log n)$ algorithms, a fact

corroborated by our results at $n = 20$ where all algorithms perform within a tight 5ms range. In modern high-performance computing, researchers like Satish [6] have pioneered GPU-based sorting that leverages thousands of cores to process billions of elements.

6. Conclusions and Future Work

This paper confirms that for large datasets, $O(n \log n)$ algorithms are strictly necessary. To address the performance degradation observed in Table 3, our next development phase will involve implementing a **Median-of-Three** pivot strategy. This involves sampling the first, middle, and last elements and using their median as the pivot, which significantly reduces the probability of encountering $O(n^2)$ behavior on sorted data.

Additionally, to support arrays of size **1,000,000**, we will refactor the Quick Sort implementation from a recursive to an **iterative** model using an explicit stack to avoid stack overflows. We also plan to explore parallelization using OpenMP to utilize multi-core architectures effectively.

References

- [1] T. H. Cormen et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [2] D. E. Knuth, *The Art of Computer Programming, Vol 3*, 1998.
- [3] C. A. R. Hoare, "Quicksort," *The Computer Journal*, 1962.
- [4] R. Sedgewick, *Algorithms*, 4th ed., 2011.
- [5] D. S. Hirschberg, "Fast parallel sorting algorithms," *ACM*, 1978.
- [6] N. Satish, "Efficient sorting algorithms for manycore GPUs," *IEEE*, 2009.

Source code and raw dataset:

<https://github.com/dragos.olaru06-creator/Academic-Writing>